Hidden Markov Model of Four Activity States

Shima Serein
Repository: github.com/supserrr/hidden-markov-models

1. Background and Motivation

A phone at the waist or on a desk streams acceleration and rotation. The use case is a campus break coach that reads this stream and decides whether the person is still, standing, walking, or jumping. Time with the phone on the desk counts as sedentary, standing in a corridor should not count as a walk across campus, and jumping counts as sport. A phone held flat while standing and a phone lying flat on a table both read about one gravity, which is why a threshold on the raw magnitude cannot separate them. I treat the four activities as the hidden states of a Gaussian hidden Markov model, with features from short windows of sensor data as its observations.

2. Data Collection and Pre-processing

Recording. I made all 200 recordings on 27 October 2026 in one session of about an hour, using Sensor Logger 1.68 on an iPhone 16 Pro that sampled the accelerometer and gyroscope every 10 ms (100 Hz). I recorded each activity as a block of 50 files: standing with the phone held steady at waist level and the screen up, walking at a steady pace with the phone held the same way, jumping continuously, and still with the phone lying on a flat surface. Files run from 5.6 to 9.9 seconds, and every activity is well over the 90-second minimum (Table 1).

Activity	Files	Total (s)	Length of one file (s)	Train / held-out files
Standing	50	438.5	5.58–9.87	40 / 10
Walking	50	393.0	7.21–8.68	40 / 10
Jumping	50	399.0	6.71–9.46	40 / 10
Still	50	438.0	7.65–9.93	40 / 10

Table 1. Recordings per activity. One file is one Sensor Logger export.

Videos. Each activity has a 5-second clip of me recording it, with my face and body in view: standing, walking, jumping, and still.

Conversion. Sensor Logger exports each recording as a zip. On iOS the accelerometer stream is user acceleration in m/s², with gravity already removed. Its spread is about 9.8 times that of the uncalibrated stream, which is in g, and that ratio confirms the unit. The gyroscope is in rad/s. src/convert.py moves the gyroscope onto the accelerometer timestamps, starts the clock at 0 s, and writes one labelled CSV per recording plus a manifest row. No file has a gap longer than 10 ms or a repeated timestamp.

Sampling rate and windows. I used one phone, which recorded at 100 Hz. To harmonise rates as the brief asks, every file is linearly interpolated onto a 50 Hz grid before windowing. For this phone that halves the rate, and a phone at any other rate would get the same 50-sample window. Windows are 1.0 second long with a 0.5-second hop. A walking step takes about half a second and I jumped about 1.5 times per second, so each window holds at least one step or jump. A one-second window also gives a native FFT bin of 1 Hz, and the chain gets a new observation twice a second.

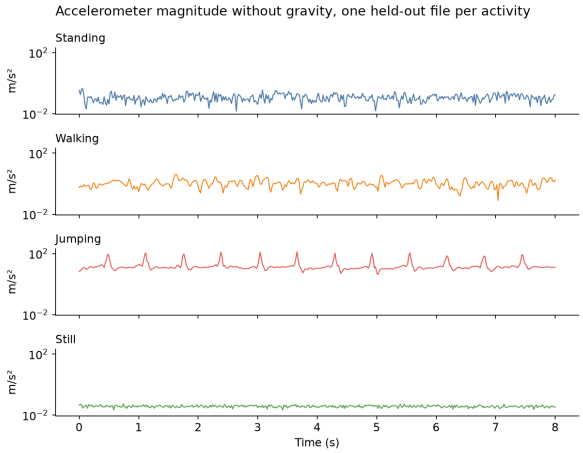


Figure 1. Accelerometer magnitude without gravity for one held-out file per activity, resampled to 50 Hz, on one logarithmic scale. Still sits at the sensor noise floor near 0.01 m/s², standing sways around 0.1, walking moves around 1, and every jump lands as a spike near 100 m/s².

Held-out and unseen data. I kept two test sets out of training. The held-out set is the last 10 recordings of each activity, made after its training files. The unseen set is a second session on the same phone 11 hours later (22:29 to 22:36 UTC), with 5 single-activity files per activity and 2 continuous recordings of about 61 s in which I changed activity three to five times (marked in data/unseen/segments.csv). Windows where I put the phone down or picked it up are labelled transition and not scored. The model is fitted on the other 40 files per activity (2,467 windows), and I tuned nothing on either test set.

Features and normalisation. Each window gives nine features (Table 2). Subtracting the window's mean acceleration first removes the phone's tilt and leaves its motion. Six features are time-domain and three come from an FFT of the window, zero-padded to eight times its length. The six size features span about seven orders of magnitude here, with a training mean spectral energy of 0.0008 for still and 13,454 for jumping. Over that range a z-score is set by jumping alone and puts standing and still closer together than one Gaussian's variance floor, so the size features go through log10 first. Every feature is then z-scored with the mean and standard deviation of the training windows, and the test windows reuse those values.

Feature	Domain	Why it is in the model (training means)
log acc_mag_mean	Time	Average motion size: 0.009 m/s² still to 13.3 m/s² jumping.
log acc_mag_var	Time	Spread of the magnitude: a landing is a spike, sway is not.
log acc_mag_rms	Time	Energy of the magnitude: separates a hard landing from a step.
log sma	Time	Signal magnitude area: motion on any axis counts.
acc_xy_corr	Time	Horizontal-axis correlation: about 0 at rest, 0.35 to 0.41 moving.
acc_dom_freq	Frequency	Busiest-axis FFT peak; fires in only 27% of walking windows.
log acc_spectral_energy	Frequency	FFT power from 0.15 to 8 Hz: the widest gap between the classes.
log gyro_mag_mean	Time	Rotation: 0.001 rad/s on the desk, 3.3 rad/s jumping.
gyro_dom_freq	Frequency	Gyroscope FFT peak: fires in 86% of jumping windows, near 1.5 Hz.

Table 2. The nine features. "log" marks the six that go through log10 before the z-score.

3. HMM Setup and Implementation

The hidden states Z are standing, walking, jumping, and still, and an observation X is the nine-dimensional feature vector of one window. The transition matrix A gives the probability of moving between states from one window to the next, and the initial vector π gives the probability of the first state. The emission model B gives each state a diagonal Gaussian over the scaled features. A diagonal covariance is enough once the features are scaled, and it needs 18 parameters per state instead of 54.

I wrote everything in NumPy (src/hmm.py). Baum-Welch learns π, A, and the Gaussian means and variances. The forward and backward passes work in log space to avoid underflow on long files, and floors of 10^{-3} on variances and 10^{-8} on transitions keep every log-probability finite. Training stops when the relative change in log-likelihood drops below 10^{-6}, which took 4 iterations here, from 9,189.7 to 9,471.1, with no decrease along the way. A self-test on a two-state chain with known parameters recovers the true path with accuracy 1.00 and checks that the posteriors sum to one.

The parameters start from the labelled training windows. Each state's Gaussian is fitted to its own windows, and every transition count gets a Dirichlet count of one so that a change never seen in training stays possible. After EM I matched states to labels using the training files only, and EM had kept each activity in its own state.

Algorithm choice. I use Baum-Welch for learning because a deployed phone will not have labels, and EM can keep adapting π, A, and B from unlabelled windows. I use Viterbi for decoding because sensitivity, specificity, and accuracy count one hard label per window, and the Viterbi path is the most likely sequence of those labels, which is what a live display would show. Forward-backward posteriors drive the E-step and would suit a display that shows uncertainty, but the metrics here score hard labels.

4. Results and Interpretation

On the held-out set the Viterbi path is right on all 581 windows and all 40 files. The labelled starting point already scores 1.000 there, and EM changes no label. Table 3 covers the unseen session, counting one scored window as one sample. The model gets 5 of 486 windows wrong, an accuracy of 0.990 (0.992 on the single-activity files and 0.987 on the continuous recordings).

State (activity)	Number of samples	Sensitivity	Specificity	Overall accuracy
Standing	104	0.981	0.995	0.990
Walking	147	0.993	0.994	0.990
Jumping	124	1.000	0.997	0.990
Still	111	0.982	1.000	0.990

Table 3. One-versus-rest sensitivity and specificity on the unseen session (20 files, 2 continuous recordings). Overall accuracy is the same quantity on every row.

On every size feature in Figure 2 the means run still, standing, walking, jumping, about 0.9 pooled standard deviations apart, while no state's own spread exceeds 0.14. That gap is why decoding is nearly exact. The frequency features help less. Their spreads are wide, and the acceleration peak appears in only 27% of walking windows because a one-second window holds only about two strides.

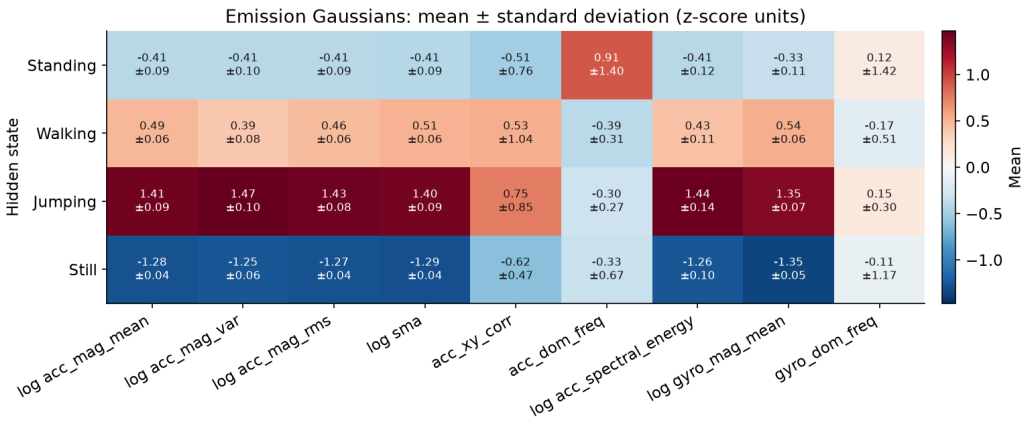


Figure 2. Emission model B: mean and standard deviation of each state's Gaussian on every scaled feature.

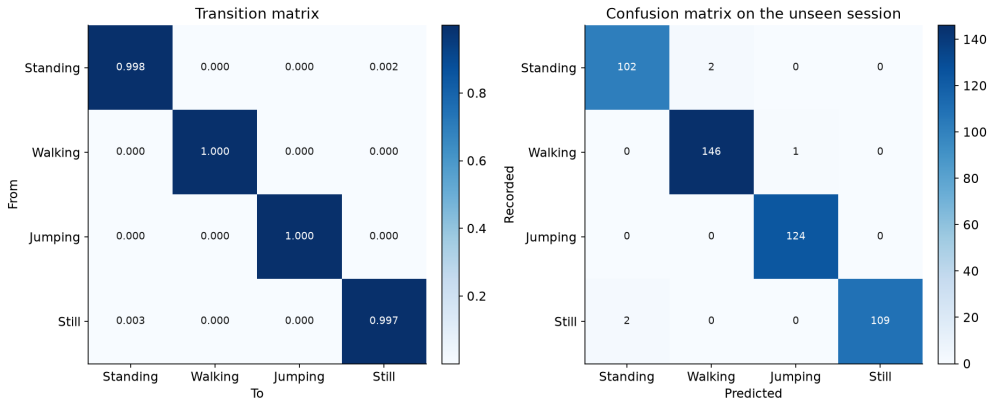


Figure 3. Left: learned transition matrix A; each row sums to 1. Right: confusion matrix on the 486 unseen-session windows.

The transition matrix is close to the identity, with self-transitions of 0.998 for standing, 0.997 for still, and 1.000 for walking and jumping. With a half-second hop, a self-transition a gives an expected stay of 0.5 / (1 − a) seconds. That is about 5 minutes for standing and 2.5 minutes for still, which is plausible, but it also predicts walks and jumps that never end. The cause is the recording protocol. Each training file holds one activity, so every pair of consecutive training windows is a self-transition. The small standing-to-still and still-to-standing probabilities (0.002 and 0.003) come from EM treating quiet standing windows as partly still.

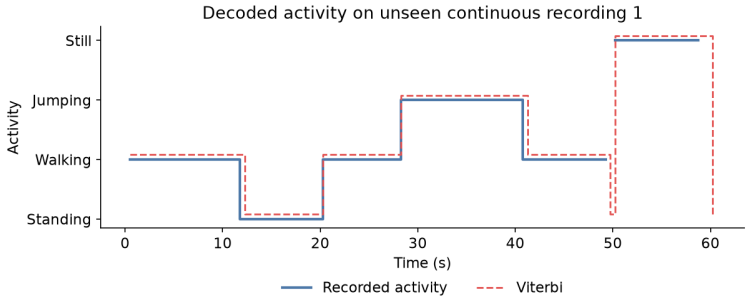


Figure 4. Viterbi on the first unseen continuous recording: walking, standing, walking, jumping, walking, still. Gaps in the recorded line are the unscored transitions.

Figure 4 is a real recording in which I changed activity five times, and Viterbi follows every change to within one window. It switches even though A gives each change a probability near 10^{-8}, because the emissions of neighbouring states differ by far more than that penalty. Three of the five wrong unseen windows straddle a change and hold half of each activity. The other two are the first second of one still file, when the phone had not yet settled after I put it down.

5. Discussion and Conclusion

Easiest and hardest activities. Jumping was the easiest state to recognise. Its spectral energy is about 700 times that of walking, and its gyroscope peak appears in 86% of its windows. Walking came next. Standing and still were the hardest pair, because the phone was flat and screen-up in both and only the size of the motion differs, about 0.1 m/s^2 of sway against 0.01 m/s^2 of sensor noise. With a plain z-score that difference is smaller than the variance floor, and Baum-Welch merged the two states. Accuracy fell to 0.862, standing sensitivity to 0.587, and the model learned a standing-to-still probability of 0.52 that no recording supports. Taking the log of the size features first keeps the two states apart.

Generalisation. The model scores 0.990 on a session recorded 11 hours later and follows activity changes it never saw in training, so it does generalise from one session to the next. Both sessions used the same phone held the same way, though, so the test says nothing about a phone in a pocket, a different surface, or another person's gait.

Sensor noise and sampling rate. Sensor noise puts the still state at about 0.01 m/s^2, ten times below standing sway, and caused no errors here. The 100 Hz stream itself was clean. My downsampling to 50 Hz has no low-pass filter, which lets energy from sharp landings above 25 Hz fold into the band below; filtering before resampling would fix that. The window length mattered more than the sampling rate. One second is too short for a reliable cadence peak, and a two-second window roughly doubles the share of walking windows with a peak, from 27% to 55%, at the cost of reacting more slowly to a change.

Improvements. The continuous recordings currently only test transitions. Training on more of them would let A learn how people actually move between activities instead of copying the recording protocol. Next would come a second participant and phone, a pocket position, the gravity direction as an extra feature, and a Dirichlet prior kept in every M-step so that a rare change cannot fall to the 10^{-8} floor.

Conclusion. A Gaussian HMM written from scratch with log-scaled motion features separates the four activities on both test sets, scoring 1.000 on the held-out files and 0.990 on a session recorded 11 hours later, where Viterbi also tracked real changes of activity. Its main limits are that all the data came from one phone and that the transition matrix was learned only from single-activity files.